Student Name : Maneesha Eeshwara – 300392759

Project : FarmVendor - Smart Supply Chain Management Platform

Course : W26-4495

Reporting Period : 03-Mar-2025 to 09-Mar-2025

Progress Report #3

Work Date/Hours Log

Date	No of Hours	Description of work done
03-Mar	6	Implemented inventory lot creation and stock validation logic to support dispatch planning from farmer inventory against vendor demand requests.
		Developed Vendor Dashboard functionality for Create Demand Request and integrated frontend actions with backend API endpoints.
		Implemented Incoming Dispatches view and Confirm Delivery workflow, including updates to dispatch delivery status and related demand request status.
05-Mar	5	Resolved full-stack integration issues involving DTO mismatches, dropdown binding problems, missing response fields, and EF Core query typing issues.
		Updated VendorDashboard.jsx and integrated modal-based demand request creation with VendorDemandRequestsController.cs.
		Added backend support for CSV/data extraction to prepare for forecasting and experimental analysis.
07-Mar	6	Began implementation of a moving-average demand forecasting baseline in .NET as the first forecasting approach for the research module.
		Created the DemandForecast model, supporting DTOs, and required database updates for forecast storage.
		Completed baseline forecasting workflow and tested forecast generation successfully through API endpoints.
		Designed the optimization module structure by defining the optimization result table and dispatch recommendation flow.
		Implemented DispatchOptimizationService.cs using greedy baseline allocation logic based on forecast demand and available farmer inventory.
		Created optimization API endpoints for generating and retrieving recommended dispatch plans.

08-Mar	3	Conducted research and design for the optimization component, focusing on demand-driven farmer-to-vendor dispatch decision logic using forecast and inventory data.
		Refined the optimization formulation to support the research novelty component of the project.
09-Mar	7	Implemented the optimization module and connected it with forecast and inventory data to generate recommended dispatch plans.
		Implemented relationship score tracking between Farmer and Vendor pairs based on dispatch outcomes.
		Added automatic score updates for Delivered and Cancelled dispatch events.
		Computed relationship score using measurable factors including on-time delivery rate, fulfillment rate, and cancellation rate.
		Exposed backend API endpoints for viewing relationship statistics.
		Added and updated files required for relationship scoring, including RelationshipStat, RelationshipScoreRowDto, RelationshipScoreService, and RelationshipStatsController.
		Integrated score update logic into VendorDispatchesController confirm delivery flow and outlined remaining frontend analytics views for forecast, optimization, and relationship endpoints. Developed Progress report3

Summary Description of work done during this reporting period

This week, I expanded FarmVendor system beyond its initial MVP and into the advanced modules required for forecasting, optimization, and relationship analytics.

I implemented full vendor-side functionality by creating inventory lots, validating stock, enabling farmers to dispatch products accurately based on vendor demand requests. The vendor dashboard expanded by adding functionality for creating demand request,created separate navigations for incoming dispatches, Confirm Delivery workflow.This screen allows vendors to acknowledge deliveries and automatically updates both the dispatch status and related demand request status. During this process, several full-stack integration issues were identified and resolved, including DTO mismatches, dropdown binding errors, missing API fields, and EF Core type-mapping problems

I developed a baseline demand forecasting module using a simple moving-average method implemented entirely in .NET and saved data in a separate table for later evaluation. Then I developed optimization module and matched inventory to predict vendor demand using greedy allocation approach. This workflow exposed through new API end points and tested and confirmed on postman while sorting different types of authorization issues using the support of AI tools.Finally I developed relationship score module to evaluate long term performance between farmers and vendors.Th escore system measure fullfilment rate, on-time delivery and cancellation frequency which fully integrated with delivery confirmation flow that can automatically scores whenever a dispatch is marked delivered or cancelled.I added API end points to retrieve relationship statistics whick confirmed via postman.

Next Step :

1. Create **frontend analytics pages** to display forecasting results, optimization plans, and relationship scores.
2. Connect frontend views to backend APIs:
    a. GET /api/forecasts
    b. POST /api/forecasts/generate
    c. GET /api/optimization/plans
    d. POST /api/optimization/run
    e. GET /api/relationshipstats
3. Perform **forecasting evaluation**
4. Evaluate **optimization results** using metrics like demand fulfillment rate, inventory utilization, and delivery efficiency
5. Analyze **relationship score trends** between farmers and vendors based on delivery outcomes.
6. Prepare **graphs and tables** for experimental results.
7. Document findings and integrate them into the **final project report and presentation**

Repo Check in of Implementation completed

The implementation work completed during this reporting period has been periodically checked into the GitHub repository under the Implementation folder. The files and folders I have checked into the repository are as follows:

Backend Implementation (files added/updated):

- VendorDemandRequestsController.cs
- VendorDispatchesController.cs
- ForecastController.cs
- OptimizationController.cs
- RelationshipStatsController.cs
- DispatchOptimizationService.cs
- DemandForecastService.cs
- RelationshipScoreService.cs
- DemandForecast.cs
- RecommendedDispatchPlan.cs
- RelationshipStat.cs
- GenerateForecastDto.cs
- RunOptimizationDto.cs
- DemandForecastRowDto.cs
- RecommendedDispatchPlanRowDto.cs
- RelationshipScoreRowDto.cs
- Updates to existing models:
    - Dispatch.cs
    - DemandRequest.cs
    - InventoryLot.cs
- EF Core migrations for:
    - Forecasting tables
    - Recommended dispatch plan tables
    - Relationship statistics tables

Frontend Implementation(files added/updated):

- Updates to VendorDashboard.jsx to integrate
    - Create Demand Request
    - Incoming Dispatches view
    - Confirm Delivery workflow
- UI logic updates for dispatch confirmation and forecasting/optimization integration.
- Dashboard layout and interaction improvements.
- Styling and minor UI enhancements to support new workflows

All changes have been committed to the repository under the appropriate Implementation directory. The progress report document will be checked into the DocumentsAndReports folder as required, and submitted on Blackboard.

DocumentsAndReports

- Progress reports3 document
- Worklog excel sheet
- Installation steps document
- Issues identified document
- Appendix

AI Use Section

Date	Work Completed	Details / Description
03-Mar	Vendor-side functional enhancements	Implemented inventory lot creation and stock validation, enabling farmers to dispatch products based on vendor demand requests. Integrated Create Demand Request UI with backend APIs. Built Incoming Dispatches view and the Confirm Delivery workflow, which updates dispatch and demand request statuses automatically. Fixed multiple integration issues including DTO mismatches, binding errors, missing fields, and EF Core query problems. Added backend CSV/data export features.
07-Mar	Baseline demand forecasting module	Developed a simple moving-average forecasting engine in .NET. Added logic to generate product-level demand predictions and store them in the database for later research and evaluation.

March 7–8	Dispatch optimization module	Designed and implemented a greedy allocation optimization technique to match farmer inventory with forecasted vendor demand. Created the DispatchOptimizationService, integrated it with inventory and forecast data, and added API endpoints for generating and retrieving optimized dispatch plans.
08-Mar	Optimization research & design	Conducted research on allocation strategies to determine how forecasted demand and inventory should drive decisions on which farmer supplies which vendor and in what quantity. Refined the optimization workflow based on analysis.
09-Mar	Relationship scoring module	Implemented a scoring model that evaluates farmer–vendor collaboration based on fulfillment rate, on-time delivery, and cancellation rate. Added models, DTOs, and API endpoints for viewing relationship statistics. Integrated automatic score recalculation into the delivery confirmation workflow so that scores update whenever a dispatch is delivered or cancelled.
March 3–9	Full-stack integration & research preparation	Resolved backend–frontend mismatches across DTOs, route models, and API responses. Ensured that forecasting, optimization, and scoring modules are fully operational for the upcoming experimental evaluation phase. Added database logging and support structures for future analysis.

	Developed worklog document in professional way	Added required file names accordingly with work done

Value Added Beyond AI

- Integrated all AI suggested logic into an existing .NET 8 + Identity architecture.
- Independently designed and finalized the database schema relationships and entity models, with the correct mapping between Farmers, Vendors, Products, InventoryLots, DemandRequests, Dispatches, Forecasts, and Relationship statistics.
- Identified and debugged integration issues specific to EF Core, API routing, and JWT authentication
- Implemented real database persistence using Entity Framework Core and SQL Server, ensuring that all forecasting, optimization, and relationship tracking modules operate on actual stored data.
- Developed and enforced custom business logic rules, including stock validation during dispatch creation, demand request matching, delivery confirmation workflows, and relationship score updates.
- Performed below complete frontend–backend integration using React and ASP.NET Core APIs
    - building dashboard views
    - implementing API calls
    - validating request payloads
    - testing endpoints through Postman and the web interface.

- Designed and implemented below core research components
    - a baseline demand forecasting module
    - a dispatch optimization module

- a relationship scores tracking system that evaluates farmer–vendor collaboration performance.

Appendix – Prompt History

Below are representative prompts used during this reporting period:

1. “How to implement a moving-average demand forecasting module in ASP.NET Core using EF Core data”
2. “Design the DemandForecast model and API endpoints for generating and retrieving forecast results”
3. “How to create a greedy optimization algorithm to allocate farmer inventory to vendor demand requests”
4. “How to design RecommendedDispatchPlan table and DispatchOptimizationService for dispatch recommendation logic”
5. “How to implement relationship score tracking between farmers and vendors based on delivery outcomes”
6. “How to update relationship scores automatically when dispatch status changes to Delivered or Cancelled”
7. “Fix dependency injection error when using RelationshipScoreService in ASP.NET Core controller”
8. “Create React frontend views to display forecast results, optimization plans, and relationship statistics”
9. “How to integrate backend analytics endpoints with React dashboard pages using secure API calls”
10. “How to export database data to CSV for research experiments and evaluation metrics”

All prompts were used for debugging assistance and architecture clarification only. Final implementation logic was customized and tested independently.